

Scaling Drifting VQ-VAE *to Real Images*

Can a physics-inspired fix for codebook collapse survive the jump from 2D toys to deep CNNs on natural images?

Ethan Zhang ez26@stanford.edu

Hemal Arora hemal1@stanford.edu

Tesvara Jiang tesvaraj@stanford.edu

How the quantizer works

Same autoencoder, swap the middle

A CNN encoder maps an image to an 8×8 grid of latent vectors. Each vector is snapped to its nearest entry in a learnable codebook of K vectors; a CNN decoder rebuilds the image. DriftingVQ returns quantized, indices, loss just like vanilla VQ, so we just swap this out.

What changes: the loss

Vanilla VQ trains the codebook with a codebook + commitment loss. DriftingVQ throws those out and replaces them with a single physics-style potential energy. Encoder outputs are treated as positive charges, codewords as negative charges, and the system just rolls downhill in energy.

Pairwise potential

$$U(r) = \tau (r + \tau) e^{-r/\tau} q_1 q_2$$

U_{pn} hidden ↔ code

attraction — pulls codes onto data · **cheap** $N \times K$

U_{nn} code ↔ code

repulsion — spreads codes apart · **cheap** $K \times K$

U_{pp} hidden ↔ hidden

repulsion between data points · **costly** N^2

Total energy $E = U_{pn} + U_{nn} + U_{pp}$ *added to the reconstruction MSE.*

In one line: opposite charges attract so dead codes get revived; same charges repel so everything spreads out evenly.

Why charges should help

Three design choices, and the reason behind each

Charges that sum to zero

Every encoder output carries $+1/N$, every codeword $-1/K$. All hiddens sum to $+1$, all codes to -1 — a balanced, electrically neutral system, so it settles at a stable equilibrium instead of blowing up.

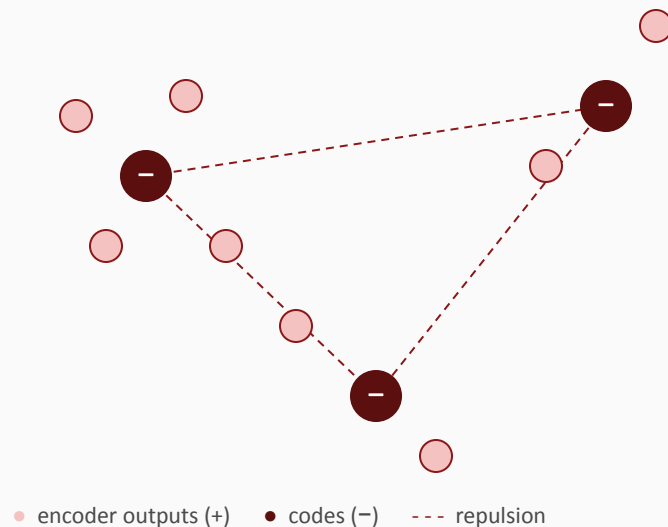
τ sets the interaction range

τ is how far a charge is felt. With L2-normalisation on (default), every vector sits on a unit sphere so distances live in $[0, 2]$ and τ means the same thing on every run.

Rotation trick for gradients

The path back to the encoder uses the rotation trick rather than the plain straight-through estimator — a smoother gradient that the SimVQ example already relies on.

Latent space (schematic)



THE BET vs BASELINES

Vanilla only updates the nearest code, so far codes starve. SimVQ shares one linear layer across all codes. Drift instead pushes every code with a global force field — so it should spread the codebook more evenly than either.

Our experiment details

Three quantizers, one shared autoencoder

vanilla

Classic nearest-neighbour VQ (van den Oord, 2017). The standard baseline.

simvq

SimVQ (Zhu, 2025) — reparameterises codes through a shared linear layer to fight collapse.

drift

Our DriftingVQ charge-physics quantizer (the new class we wrote).

Plus a K=2 toy: two 2D clusters, a linear encoder/decoder, 30 seeds — to unit-test drift before scaling up. Vanilla is run in both *classical* and *EMA* modes.

Training harness

Datasets	CIFAR-10 (32×32) · CelebA (64×64), both → 8×8 code grid
Architecture	Shared CNN enc/dec: residual blocks, GroupNorm, SiLU
Latent	dim = 64 · hidden width = 128
Codebook sizes	K = 64, 256, 512, 1024
Optimiser	AdamW · lr 3e-4 · batch 128
Loss	reconstruction MSE + quantizer loss
Run length	2000 iters — short smoke-scale, not converged
Metrics	PSNR · perplexity · utilization · (→ SSIM, FID)

Reported per (method, dataset, K): codebook utilization, perplexity, and PSNR — the same three numbers everywhere, so methods line up cleanly.

Our Metrics

Perplexity ^e

Number of codewords effectively used · **Max K**

Perplexity note: this is the opposite of what we want for LLMs, because here we want a more spread out distribution. Defined probabilistically (i.e. in terms of entropy).

PSNR (dB)

Peak Signal-to-Noise Ratio in decibels · $(-\infty, \infty)$

Utilization Percentage

Number of codewords used at least once · **[0, 100]**

Utilization Percentage note: unlike perplexity, this classifies codewords in a binary fashion. This can lead to this metric distinguishing VQs at a smaller margin.

For all three metrics, higher > more performative.

Initial run results for CIFAR-10/CelebA

TOY K=2 · collapse over 30 seeds

baseline used	vanilla	drift
classical vanilla	97% (29/30)	17% (5/30)
EMA vanilla	3% (1/30)	17% (5/30)

Drift crushes the *classical* baseline (Liu's headline claim reproduced), but not vs modern EMA vanilla which barely collapses on its own.

CODEBOOK USAGE · perplexity (higher = better)

Drift has the highest perplexity at every K on both datasets, with ~100% utilization. SimVQ is the opposite — at K=64 it effectively uses only ~8 of 64 codes.

CIFAR K=1024 perplexity: **drift 758.6** > **vanilla 678.6** » **simvq 54.0**

PSNR by K · CIFAR-10 (dB, higher = better)

method	64	256	512	1024
vanilla	22.26	23.42	23.81	24.37
simvq	19.10	20.03	20.49	20.94
drift	20.87	21.71	22.12	22.43

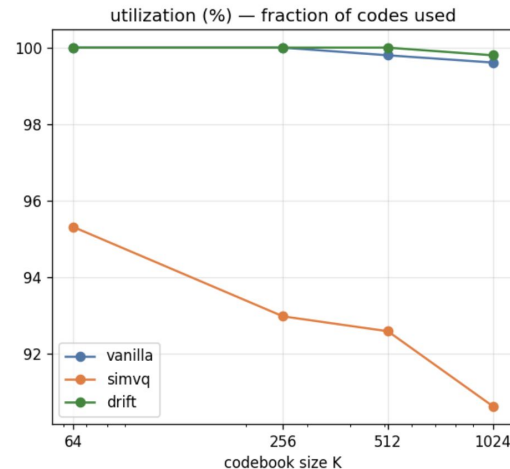
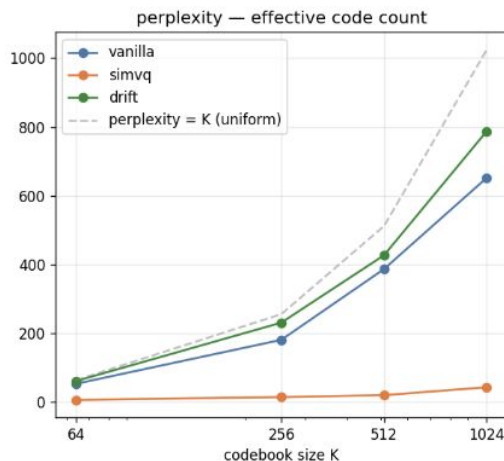
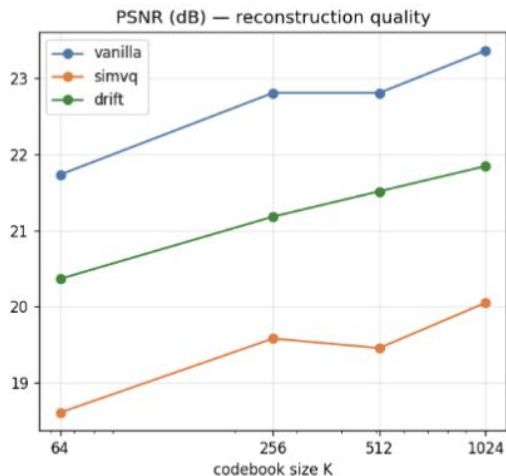
PSNR by K · CELEB-A

method	64	256	512	1024
vanilla	21.74	22.81	22.81	23.37
simvq	18.60	19.58	19.45	20.05
drift	20.36	21.18	21.51	21.85

Our Initial Hypothesis: drift spreads the vocabulary best, yet vanilla still rebuilds the picture ~1.5–1.9 dB sharper. Using the codebook well ≠ placing it well.

*** Also, **energy_weight** is a soft dial: 0.1→4.0 raises perplexity but trades ~0.2 dB PSNR. Lower looks best.*

CelebA K-Sweep Runs Results



Plotting CelebA Results: vanilla wins reconstruction by ~1.4–1.5 dB (left), drift wins on perplexity on every K (middle), and drift wins usage at every K > 256 (right), simvq collapses.

Our plan from here...

Currently, we find:

✓ DELIVERS ON ITS PROMISE

Drift spreads codebook usage more evenly than anything else, highest perplexity and near-perfect utilization at every K on both datasets. No dead codes. SimVQ, by contrast, collapses badly.

⚠ SPREAD ≠ SHARPER IMAGES (yet)

Drift reconstructs ~1.5–2 dB worse than vanilla. Vanilla's usage is already ~100%, so drift's extra even-ness isn't causing a reconstruction win.

⚠ THE COLLAPSE-FIX IS BASELINE-DEPENDENT

Drift beats classical vanilla 97%→17%, but modern EMA vanilla barely collapses anyway.

1 Longer runs

2000 iters is nowhere near converged. Train far longer to see if drift's codebook spread eventually pays off in PSNR.

2 Add SSIM + FID

PSNR only measures pixel error. Add perceptual metrics — FID may reward drift's even coverage in a way PSNR doesn't.

3 Fair-baseline framing

Always report drift against both classical and EMA vanilla, so the collapse claim is honest.

4 Probe the gap

Diagnose why even usage doesn't translate to reconstruction — is it placement, the rotation-trick gradient, or T tuning?

BOTTOM LINE

The method is implemented, drop-in, and reproducible across two datasets. It does exactly what the physics promises on codebook usage — the open question is turning that even coverage into sharper reconstructions.

Appendix

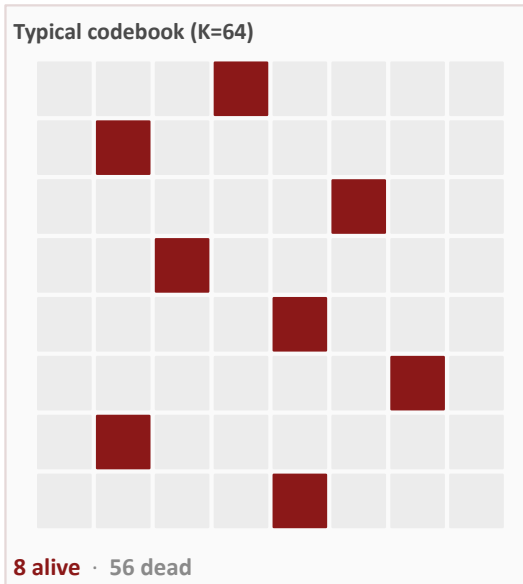
The autoencoder problem

Why VQ-VAEs matter

VQ-VAEs compress images into sequences of discrete tokens from a fixed codebook. This idea underlies many image/video tokenization pipelines, including VQGAN, DALL-E-style discrete image modeling, and VideoGPT.

The failure mode

Only the nearest codeword to each encoder output receives the direct codebook update. Codes that start far from data may rarely be selected, receive little or no useful update, and become effectively dead. In severe collapse, a large codebook can behave like a much smaller vocabulary.



Consequence: wasted model capacity, fewer effective tokens, and weaker downstream generation on top of the discrete latent space.